Software Impacts 28 (2026) 100817



Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts



Software update

Middleware-enforced Timed Causal Consistency for Apache Cassandra: An energy–performance–consistency evaluation against static consistency levels using YCSB



Hesam Nejati SharifAldin *, Farnoush Nayebi Pour

Department of Computer Engineering, Mashhad Branch, Islamic Azad University, Mashhad, Iran

ARTICLE INFO

Keywords:
Apache Cassandra
Consistency
Replication
Frequency-scaling
Energy–performance
Tools and monitoring

ABSTRACT

We present a reproducible and open-source middleware-enforced consistency model — Strict Timed Causal Consistency (STCC) — designed as an external consistency layer for Cassandra, systematically compared against built-in static levels (ALL, QUORUM, ONE) in terms of energy, performance, and consistency trade-offs under YCSB workloads. Cassandra's default consistency levels (e.g., ALL, QUORUM, ONE) are statically configured and unable to adapt to runtime variations in workload intensity or network conditions. To address this limitation, we introduce a fully functional middleware implementing *Strict Timed Causal Consistency* (STCC) — a hybrid model that combines client- and server-side causal guarantees with a configurable freshness bound Δ. STCC is non-intrusive and integrates seamlessly with unmodified Cassandra deployments, enabling adaptive consistency at runtime. The middleware integrates seamlessly with unmodified Cassandra instances, offering tunable freshness bounds (Δ) and causal guarantees across geo-replicated nodes without modifying the core database engine. The package includes automation tools for YCSB-based workload execution, CPU frequency control via `cpupower`, and energy monitoring using SNMP-enabled PDUs and `dstat`. This framework targets cloud engineers, distributed systems researchers, and middleware designers seeking to analyze or optimize consistency–latency–energy trade-offs under realistic deployment conditions. Our experiments validated on a 24-node Cassandra cluster distributed across three Cassandra's data centers, the system is publicly released with full documentation to support reproducible and extensible experimentation.

Code metadata

Current code version	v1.0.0
Source code repository	https://github.com/SoftwareImpacts/SIMPAC-2025-224
Code Ocean capsule	N/A
Legal Code License	MIT License
Code versioning system used	git
Software code languages, tools, and services used	Python 3.10, Apache Cassandra, Bash
Compilation requirements, operating environments & dependencies	cassandra-driver, six, Matplotlib, Numpy, Pip, Python ≥ 3.10
Link to documentation	https://github.com/Hesam-Nejati/STCC-Middleware/blob/main/README.md
Permanent archival link	https://doi.org/10.5281/zenodo.16742599
Support email for questions	hesam.nejati@gmail.com

1. Introduction

Distributed cloud services and social platforms such as Facebook and Google process massive volumes of real-time data, often under accurate delay and availability constraints [1,2]. Storage systems replicate data across geographically distributed nodes to meet these demands [3,4]. Replication improves fault tolerance and reduces client-perceived latency, especially when replicas are strategically placed geographically close to the users [5].

DOI of original article: https://doi.org/10.1016/j.future.2019.11.038.
* Corresponding author.
E-mail addresses: Hesam.nejatisharifaldin@ucdconnect.ie (H.N. SharifAldin), Farnoush.nayebi@mshdiau.ac.ir (F.N. Pour).

https://doi.org/10.1016/j.simpa.2026.100817
Received 16 August 2025; Accepted 1 February 2026

2665-9638/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Apache Cassandra is widely adopted in such environments for its support of horizontal scalability and tunable consistency. Its architecture leverages configurable consistency levels — (e.g., ONE, QUORUM, ALL) — to balance performance with correctness [6,7]. However, these settings are static and remain fixed during execution. As workloads shift or network latency fluctuates, the system cannot adapt. This often results in stale reads, degraded throughput, and unpredictable latency under production workloads [8,9].

The underlying challenge lies in the fundamental trade-off between consistency, availability, and convergence [10]. Stronger models such as linear or sequential consistency offer precise ordering but impose coordination overhead across replicas, degrading latency. Eventual consistency avoids coordination delays by relaxing ordering guarantees, but allows temporary divergence [11].

Causal consistency offers a middle ground. It respects happens-before relations [12] and enables concurrency without requiring global synchronization. Moreover, Causal guarantees logical ordering of operations. In addition, it does not enforce bounds on data freshness. As a result, stale but causally valid responses may occur.

Client-Centric consistency (generally speaking Session-based) guarantees — such as Monotonic Read (MR), Monotonic Write (MW), Read Your Write (RYW), and Write Follows Read (WFR) — enhance user-level consistency and are widely used in client-centric protocols [13,14]. However, these models are typically enforced at the client side and do not scale well under multi-user concurrency or cross-session interactions.

Therefore, Timed Causal Consistency (TCC) proposed a bounded freshness constraint Δ alongside causal ordering [15,16] to overcome these limitations. TCC ensures that reads are not only causally valid but also temporally recent — addressing latency-sensitive use cases where data staleness is unacceptable.

Despite these advances, prior works rarely consider the joint impact of consistency and replication strategies on system performance, and scalability [17,18].

However, an important limitation persists. Cassandra's consistency levels are statically configured at runtime and cannot adapt to fluctuating conditions, such as network delays, workload surges, or replica imbalances. As a result, the system often sacrifices either performance or data freshness. Furthermore, causal and session-based consistency models enforce logical operation order but impose no bounds on data freshness. They provide correctness by causality, yet cannot guarantee that reads reflect the most recent state. This makes them inadequate for latency-sensitive systems that demand both consistency and timely data delivery.

This work presents a fully implemented external middleware for Apache Cassandra that enforces *Strict Timed Causal Consistency (STCC)* [19], offering session consistency such as MR/MW, RYW, WFR, and a tunable freshness bound Δ to minimize read staleness across geo-replicated nodes. Unlike Cassandra's static consistency levels, STCC dynamically adapts to runtime conditions and client-observed dependencies, reducing inconsistency windows without sacrificing performance or availability.

The accompanying benchmarking suite allows systematic evaluation of Cassandra under both native consistency levels (`ALL`, `QUORUM`, `ONE`) and STCC. Users can run YCSB 0.15.0 [20] workloads to compare throughput, latency, replication behavior, and staleness mitigation under each configuration.

Beyond correctness and latency, the suite supports energy-aware benchmarking via automated CPU frequency scaling (`cpupower`) and power monitoring through SNMP-enabled PDUs and tools like `dstat`. Experiments can be configured under multiple DVFS modes such as `performance`, `powersave`, `ondemand`, `conservative`, and `userspace` and replication factors to analyze energy–performance–consistency trade-offs.

The system is validated on a 24-node Cassandra 4.0 cluster across three data centers with high-speed interconnects. Full middleware integration, workload orchestration, monitoring scripts, and reproducibility instructions are available in the open-source release.

2. Impact overview

This artifact has already been applied in published research. In particular, it was used in *Strict Timed Causal Consistency as a Hybrid Consistency Model in the Cloud Environment* [19], where the middleware enabled direct comparisons between STCC and Cassandra's static consistency levels (ONE, QUORUM, ALL) under YCSB workloads on a 24-node physical cluster. The software supported reproducible experiments and provided detailed evaluations of performance and stale-read behavior across heterogeneous cluster configurations.

From an industrial perspective, the artifact addresses a critical challenge faced by operators of No-SQL distributed database especially large-scale Cassandra deployments: balancing strong consistency guarantees with latency and energy costs. In practice, static consistency levels often lead to either unnecessary energy overheads or degraded user-facing performance, especially under replication and geo-distributed settings. The middleware provides a systematic and automated way to benchmark these trade-offs, offering practitioners a tool to set replication strategies, frequency governors, and consistency semantics for production-like workloads.

The middleware and benchmarking scripts are consolidated into a unified and reusable package in this artifact. Beyond consistency and performance, this version also incorporates *energy-aware* implementation and evaluation, capturing the impact of replication strategies, CPU frequency scaling, and consistency on Cassandra cluster operations. This allows researchers and practitioners to (i) reproduce baseline consistency experiments, (ii) assess new consistency enforcement mechanisms, and (iii) analyze energy–performance–consistency trade-offs under realistic workloads. The documented scripts and configuration files ensure portability to any Cassandra cluster, making the artifact relevant not only for academic research and distributed systems education, but also for industry practitioners and system operators who require reproducible benchmarking for consistency tuning, performance optimization, and energy-efficient cluster management.

2.1. Impact and audience

This software suite is intended to support diverse user groups working with distributed NoSQL systems, particularly Apache Cassandra. Its target audience includes:

- **Cloud database researchers** analyzing the trade-offs between consistency semantics, latency, and energy consumption under real-world workloads.
- **Middleware developers** exploring reusable and non-intrusive patterns for enforcing hybrid or client-centric consistency guarantees without modifying the database engine.
- **System architects and engineers** benchmarking performance under various replication strategies, CPU frequency governors, and consistency levels to meet energy-efficiency goals.

By combining correctness analysis (e.g., staleness, causal guarantees), performance metrics (e.g., throughput, latency), and energy measurements (via SNMP-enabled PDUs), this platform enables systematic exploration of consistency–performance–energy trade-offs in geo-distributed environments.

The software bridges the gap between theoretical models like STCC and their deployment in real-world, energy-constrained cloud infrastructures. Its deployment in a 24-node Cassandra cluster spanning three data centers demonstrates practical applicability and scalability. Furthermore, the system's reproducibility and open-source availability make it suitable for both academic research and industry benchmarking pipelines.

3. Related work

Timed consistency augments causal models with logical time to enforce operation order and data freshness in cloud storage systems [21,22]. While Causal+ ensures convergence in replicated systems like COPS by combining causal relations with conflict resolution [23], most causal models lack temporal guarantees. STCC [19] addresses this gap by integrating a tunable freshness threshold (Δ) with client-side guarantees such as Monotonic Read (MR), Monotonic Write (MW), Read Your Write (RYW), and Write Follows Read (WFR). Prior work on metadata-driven verification [14] and CRP-based protocols [24] further highlight the benefits of time-bounded causal models. Complementary to these consistency efforts, several studies have focused on energy-efficient operation of cloud-based NoSQL systems such as Cassandra. Li et al. [25] and Chihoub et al. [26] reported that exploring the trade-offs between consistency and energy can yield more than 24% energy savings, achieved through quorum-based configurations and strategic node placement. Ibrahim et al. [27] and Mahajan et al. [28] and Aldin et al. [29] applied DVFS and query-level optimizations to Hadoop, MongoDB, and Cassandra, emphasizing that energy efficiency may not always correlate with performance. Gari et al. [30] and Da et al. [17] validated Cassandra's viability under energy-constrained environments, while Gomes et al. [31,32] developed stochastic and analytical models to assess consistency–performance–energy trade-offs. Rafay et al. [33] further improved these trade-offs through synchronization enhancements using Imp-Paxos. STCC extends these research directions by unifying causal consistency with time-awareness and enabling runtime enforcement over partially replicated energy-sensitive deployments.

4. Design objectives and system architecture

4.1. System design objectives

The core design objective of STCC is to bring dynamic, fine-grained consistency enforcement into existing NoSQL infrastructures without modifying the database core. Traditional static settings in Apache Cassandra cannot adapt to real-time changes in workload, network delays, or replica skew. This leads to unavoidable trade-offs between latency, consistency, and availability.

STCC addresses these challenges through a lightweight external middleware that acts as an intelligent proxy between applications and the Cassandra backend. The middleware enables runtime adaptation based on operation dependencies and data freshness, while remaining fully compatible with Cassandra's native architecture and replication settings.

The system is built with the following design goals:

- **Adaptivity:** Enable runtime consistency enforcement that reacts to network dynamics and workload shifts.
- **Freshness:** Minimize stale reads by enforcing temporal bounds without compromising response latency.
- **Compatibility:** Preserve full compatibility with Cassandra's replication model and consistency APIs.
- **Transparency:** Avoid modifications to Cassandra's core or client libraries.
- **Scalability:** Support deployment across multi-rack, multi-datacenter configurations.

4.2. Middleware architecture and operation flow

STCC is implemented as an external, version-independent middleware layer that intercepts application-level operations, validates causal and temporal constraints, and dynamically dispatches them to the Cassandra backend. The middleware tracks each operation (`put/get`) in a distributed operation table indexed by user ID, operation type, and logical timestamp. This table forms the basis for enforcing STCC's client-side and cross-client causal guarantees.

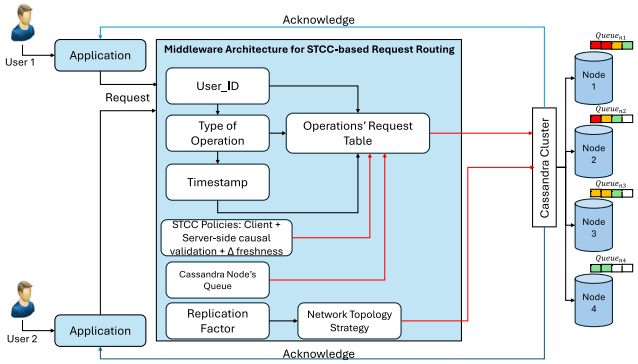


Fig. 1. STCC middleware intercepting client operations and validating temporal–causal constraints.

STCC intercepts application-level `put/get` requests and applies consistency policies dynamically at runtime. It does not modify Cassandra's internal engine.

The replication strategy is pre-configured through the `NetworkTopologyStrategy` when defining the keyspace. This allows precise control over which nodes — across clusters, racks, and data centers — receive replicated data.

The middleware continuously monitors queue depths across nodes to identify the least-loaded replica for each write request, enabling balanced dispatch and minimizing write latency. STCC applies its consistency policies by operating over the Distributed User Operation Table (DUOT), which tracks logical timestamps and dependencies.

Middleware selects the Cassandra's node with the lowest current queue length for each `put` operation. Consistency checks are performed locally on that node. The update is then propagated to other replicas according to the predefined `NetworkTopologyStrategy`, ensuring both causal validity and freshness across the cluster.

Each replica evaluates its local version against a quorum of $(\frac{\text{Replication Factor}}{2} + 1)$ nodes for each `put/get` operation. Outdated replicas reconcile their state by fetching the latest version before serving the read. This guarantees that all read operations return data that is not only causally valid but also Δ-fresh within the scope of quorum validation.

Fig. 1 illustrates clients' application submit `put/get` requests to the middleware rather than directly contacting Cassandra nodes. Upon arrival, each operation is logged into a centralized in-memory table, which records the user identifier, operation type, logical timestamp, and involved key.

The system classifies operations based on temporal locality and access origin:

- Intra-user requests are evaluated for session-level guarantees such as Monotonic Read/Write, Read-Your-Writes, and Write-Follows-Read.
- Cross-user operations are checked for causal dependencies using an on-the-fly Operation Dependency Graph (ODG) in Section 4.3.

Requests are then dispatched based on the outcome of these validations. If a read violates the configured freshness bound Δ, the system either delays the response or redirects to a more recent replica, preserving both causality and timeliness.

4.3. Consistency validation through dependency graph and Δ-freshness

Consistency model guarantees the system constructs an Operation Dependency Graph (ODG) in real time to enforce both client- and server-side. Each node in the graph represents a client-issued operation

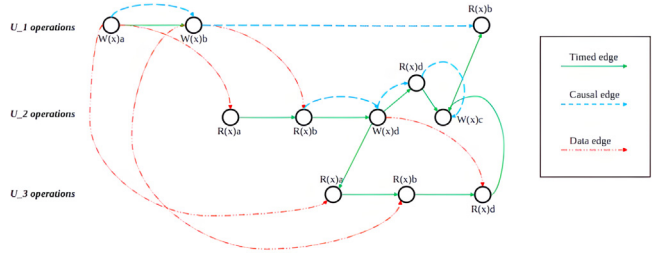


Fig. 2. An example Operation Dependency Graph (ODG) showing timed, causal, and data edges across operations from three users.

Table 1
Notation used in consistency guarantees.

Symbol	Description
$r_i^x(t)$	Read operation by client i on key x at time t
$w_i^x(t)$	Write operation by client i on key x at time t
t_1, t_2	Logical or physical timestamps, with $t_1 < t_2$
x, y	Data keys in the distributed database
Version(x, t)	The version of key x visible at time t
Visible(op)	The set of replicas where operation op is visible
$\rightarrow$	Causal ordering between operations
$\preceq$, $\succeq$	Version comparison operators (precedence and recency)
Cause(op)	Causal history of operation op

(`read/write`), annotated with the key accessed, logical timestamp, and issuing user.

Edges between operations capture three types of dependencies in Fig. 2:

- **Timed Edge** (green): Represents temporal precedence between operations issued by the same user.
- **Causal Edge** (blue dashed): Captures happens-before relationships between operations, including inter-user causal propagation.
- **Data Edge** (red dashed): Links write and read operations involving the same data version, showing value-based causality.

Fig. 2 depicts a dynamic ODG instance where three clients interact with a shared data item, enabling real-time detection of causality violations and Δ-bound breaches. The graph dynamically evolves as new operations arrive and enables efficient detection of causal violations and freshness breaches.

This structure allows the system to:

- Enforce Monotonic Reads, Monotonic Writes, Read Your Writes, and Writes Follow Reads at the client side;
- Validate Timed Causal Consistency (TCC) at the server side.

The ODG is implemented as an in-memory directed graph, updated at runtime using consistency timestamps. These timestamps simplify dependency resolution without requiring vector clocks or external coordination. The graph is simple and scales efficiently with concurrent sessions.

4.4. STCC consistency model enforcement with formal semantics

STCC enforces a set of five consistency guarantees. Each guarantee is formally defined and grounded in practical scenarios that emerge in distributed applications (see Table 1).

- **Monotonic Reads (MR):** If a client reads a value at time t_1 and later reads the same data item at $t_2 > t_1$, the second read should return a version at least as recent as the first.
  Example use case 1: A mobile app user browsing their notifications should not see older messages reappear after viewing newer ones.

$$\forall\, r_i^x(t_1),\ r_i^x(t_2)\ :\ t_1 < t_2 \Rightarrow \text{Version}(x, t_1) \preceq \text{Version}(x, t_2)$$

- **Monotonic Writes (MW):** If a client issues two write operations in order, the system must ensure the visibility of the first before applying the second.
  Example use case 2: When editing a shared document, each user edit should appear in the order it was made.

$$\forall\, w_i^x(t_1),\ w_i^x(t_2)\ :\ t_1 < t_2 \Rightarrow \text{Visible}(w_i^x(t_1)) \rightarrow \text{Visible}(w_i^x(t_2))$$

- **Read Your Writes (RYW):** A client should observe its previous write in subsequent reads of the same item.
  Example use case 3: A user posts a comment and immediately reloads the page — they expect to see their own comment.

$$\forall\, w_i^x(t_1),\ r_i^x(t_2)\ :\ t_1 < t_2 \Rightarrow \text{Version}(x, t_2) \succeq \text{Version}(x, t_1)$$

- **Writes Follow Reads (WFR):** If a write follows a read in a session, the write must preserve the causal context of the preceding read.
  Example use case 4: A user reads a shared task list and adds a new item. The addition must reflect the previously seen state.

$$r_i^x(t_1) \rightarrow w_i^y(t_2) \Rightarrow \text{Cause}(r_i^x(t_1)) \subseteq \text{Cause}(w_i^y(t_2))$$

- **Cross-User Causal Consistency with Freshness (Δ-Bound):** For different users, a read should reflect a write only if the write is causally visible and not older than a freshness bound Δ.
  Example use case 5: In collaborative editing, if user A updates a document, user B should not read a version older than Δ milliseconds from that update.

$$w_i^x(t_1) \rightarrow r_j^x(t_2) \Rightarrow \begin{cases} t_1 < t_2 \\ \text{Visible}(w_i^x(t_1)) \rightarrow \text{Visible}(r_j^x(t_2)) \\ t_2 - t_1 \leq \Delta \end{cases}$$

4.5. Runtime algorithm

Algorithm 1 describes the core logic enforcing causal and timely consistency.

Algorithm 1 formalizes the runtime enforcement of STCC within our middle-ware. It takes as input the Distributed User Operation Table (DUOT) and checks for both client-centric consistency guarantees (Monotonic Read, Monotonic Write, Read Your Write, Write Follows Read) and cross-user causal dependencies (UGD). Based on the Operation Dependency Graph (ODG), the algorithm determines the correct order of execution and dispatches operations to Cassandra nodes according to queue size and replication constraints. Reads are verified against freshness threshold Δ using quorum comparison. This mechanism enables dynamic consistency enforcement without modifying Cassandra's core.

5. Experimental setup and evaluation metrics

The experimental cluster comprised 24 physical nodes distributed across three data centers (DCs), with each DC containing two racks and four nodes per rack, as shown in Fig. 3. Each node ran Cassandra 4.0, with hardware specifications including an 8-core Intel Xeon E5-2650 CPU (2.8 GHz max), 16 GB RAM, and 512 GB SSD storage.

Networking was provisioned with 1 Gb Ethernet for control plane communication and 20 Gb Infiniband interconnects for data-intensive replication and coordination. Cassandra's NetworkTopologyStrategy was configured to reflect the multi-DC, multi-rack topology, enabling replica placement across racks and ensuring fault tolerance.

We defined a dedicated keyspace called *user_network_topology* using Cassandra's `NetworkTopologyStrategy` to support realistic consistency enforcement across geographically distributed nodes. Fig. 4 shows this configuration allows fine-grained control over replica placement across three data centers. Each data center hosts a different number of replicas based on hardware availability and projected workload.

Algorithm 1: Strict Timed Causal Consistency (STCC)

Input : Distributed User Operation Table (DUOT)
Data: O_1, O_2: operations
u: user ID
x: data item
ts: timestamp
Q: operation queue
RF: replication factor
Δ: freshness threshold

while *there is new* O_i *on DUOT* **do**
  Insert O_i into Operations Table with $[UserID, Type, ts]$;
  if $O_1 \rightarrow O_2$ *and* $User(O_1) = User(O_2)$ **then**
    // Enforce client-centric guarantees
    if $ODG \vDash O_1 \prec_t O_2$ **then**
      if $ODG \vDash O_1 \prec_d O_2$ *or* $O_1 \prec_c O_2$ **then**
        Monotonic Read is satisfied;
    if $ODG \vDash W(x)_1 \prec_c W(x)_2$ *and* $ts_1 < ts_2$ **then**
      Monotonic Write is satisfied;
    if $ODG \vDash W(x)_1 \rightarrow R(x)_2$ **then**
      Read Your Write is satisfied;
    if $ODG \vDash R(x)_1 \rightarrow W(x)_2$ **then**
      Write Follows Read is satisfied;
  else
    // Check for User-Generated Dependencies (UGD)
    if $ODG \vDash O_1 \prec_t O_2$ *and* $O_1 \prec_c O_2$ **then**
      Causal Consistency required;
  // Dispatching based on queue length
  Select $Node_i$ with minimal $|Q|$;
  // Write Operation Replication
  if O_i *is Write* **then**
    Replicate to RF nodes based on Network topology strategy;
  // Read Operation Freshness
  if O_i *is Read* **then**
    Query any node;
    Compare local version with quorum ($\geq \frac{RF}{2} + 1$);
    if *Version is stale* **then**
      Fetch latest version;
    if *Version age* $\leq \Delta$ **then**
      Return result to user;
    else
      Defer until fresh replica available;

This setup enables the middleware to test consistency coordination under practical deployment conditions while reducing inter-datacenter communication overhead. It also ensures that STCC can balance latency and causality across asymmetric infrastructure layouts.

Fig. 5 displays all active keyspaces in the Cassandra cluster. Among them, *user_network_topology* was introduced specifically for multi-datacenter consistency experiments. As detailed in Fig. 4, this keyspace applies custom replication factors across three data centers to simulate realistic conditions for evaluating STCC coordination policies.

Fig. 6 shows the creation of the `ycsb` table within the *user_network_topology* keyspace. The schema follows YCSB's default format, where *y_id* is the primary key and fields `field0` through `field9` store record values. This structure was used to initialize the dataset before executing benchmark workloads.

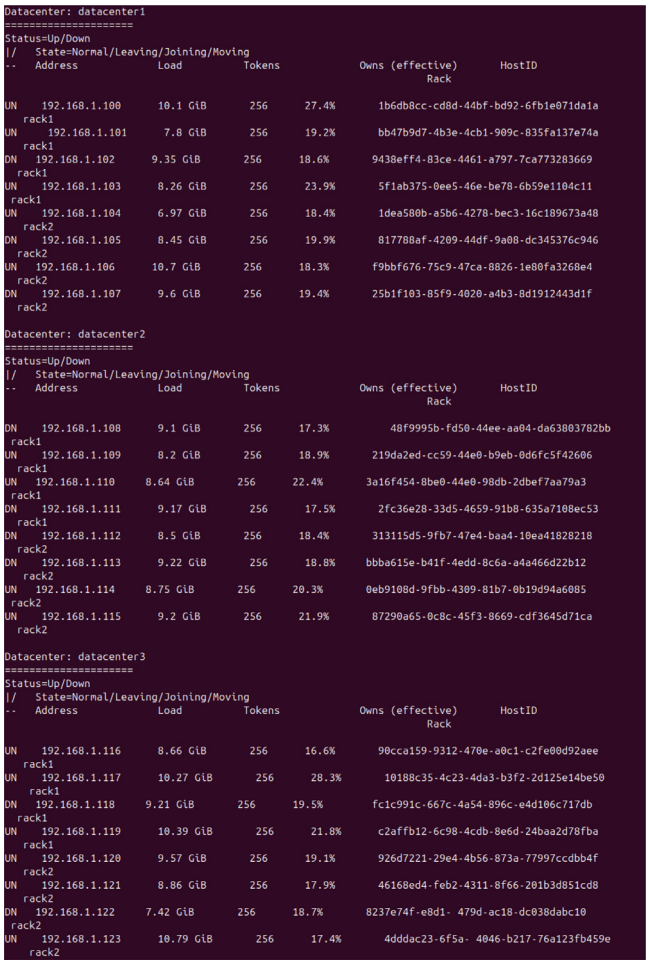


Fig. 3. Cluster topology captured via `nodetool status`. Each DC consists of 2 racks with 4 nodes each.

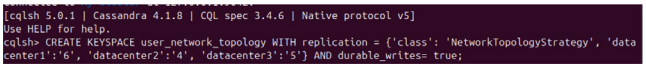


Fig. 4. Keyspace definition using `NetworkTopologyStrategy` to enable customized replication across three datacenters. This setup ensures that write and read operations are consistently distributed across **three data centers** — `datacenter1`, `datacenter2`, and `datacenter3` — with **6**, **4**, and **5** replicas respectively. The configuration enables STCC to enforce cross-datacenter consistency constraints while respecting physical network topology and fault domains.

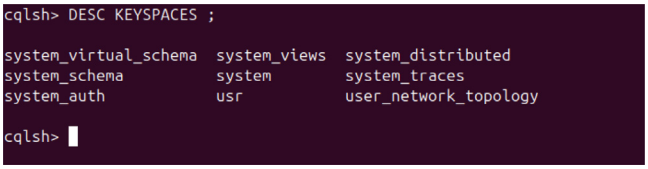


Fig. 5. Defined keyspaces in the Cassandra cluster, including the *user_network_topology* keyspace used for enforcing distributed consistency policies.

5.1. Workload generation

Workloads were generated using YCSB 0.15.0 with profiles:

- Workload A: 60/40 read/write

```
cqlsh> CREATE TABLE IF NOT EXISTS  user_network_topology.ycsb (
   ... y_id varchar PRIMARY KEY ,
   ... field0 varchar,
   ... field1 varchar,
   ... field2 varchar,
   ... field3 varchar,
   ... field4 varchar,
   ... field5 varchar,
   ... field6 varchar,
   ... field7 varchar,
   ... field8 varchar,
   ... field9 varchar);
cqlsh> SELECT * FROM user_network_topology.ycsb ;

 y_id | field0 | field1 | field2 | field3 | field4 | field5 | field6 | field7 | field8 | field9
------+--------+--------+--------+--------+--------+--------+--------+--------+--------+--------

(0 rows)
cqlsh>
```

Fig. 6. Schema definition of the `ycsb` table inside the *user_network_topology* keyspace. This table was initialized prior to workload execution.

```
GNU nano 7.2                                  workloada *
# Copyright (c) 2010 Yahoo! Inc. All rights reserved.
#
# Licensed under the Apache License, Version 2.0 (the "License"); you
# may not use this file except in compliance with the License. You
# may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
# implied. See the License for the specific language governing
# permissions and limitations under the License. See accompanying
# LICENSE file.


# Yahoo! Cloud System Benchmark
# Workload A: Update heavy workload
#   Application example: Session store recording recent actions
#
#   Read/update ratio: 50/50
#   Default data size: 1 KB records (10 fields, 100 bytes each, plus key)
#   Request distribution: zipfian

recordcount=10000
operationcount=10000
workload=site.ycsb.workloads.CoreWorkload

readallfields=true

readproportion=0.6
updateproportion=0.4
scanproportion=0
insertproportion=0

requestdistribution=zipfian
```

Fig. 7. Configuration of *workload_a*: a workload with 60% reads and 40% updates. This setup simulates applications such as session stores with equal probability of read and write operations.

- Workload B: 95/05 read-heavy
- Workload C: 100% read

Figs. 7, 8, and 9 illustrate the configuration files used for three YCSB workloads. These files are located in the YCSB/workloads/ directory and define the read/write behavior, operation counts, and access distributions for benchmarking.

Each configuration defines recordcount=1000 and operationcount =1000, ensuring uniform benchmarking across all workloads. Fields are accessed entirely (readallfields=true), and a Zipfian distribution is used to emulate non-uniform key access patterns common in real deployments.

Unlike standard YCSB benchmarks that connect directly to the database, our setup introduces an intermediate coordination layer — STCC middleware — which intercepts every read and write operation. Before reaching Cassandra, each YCSB request is evaluated for its consistency requirements. The middleware inspects whether the request belongs to a same-user sequence or a cross-user causal path, using the Operation Dependency Graph (ODG) and User-Generated Dependencies (UGD). Based on this analysis, it dynamically enforces one of five consistency guarantees, then forwards the operation to the target table in the *user_network_topology* keyspace. This architecture ensures that consistency validation precedes all interactions with the backend, enabling strict and tunable coordination without modifying the Cassandra source code.

```
GNU nano 7.2                                  workloadb
# Copyright (c) 2010 Yahoo! Inc. All rights reserved.
#
# Licensed under the Apache License, Version 2.0 (the "License"); you
# may not use this file except in compliance with the License. You
# may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
# implied. See the License for the specific language governing
# permissions and limitations under the License. See accompanying
# LICENSE file.

# Yahoo! Cloud System Benchmark
# Workload B: Read mostly workload
#   Application example: photo tagging; add a tag is an update, but most operations are to read tags
#
#   Read/update ratio: 95/5
#   Default data size: 1 KB records (10 fields, 100 bytes each, plus key)
#   Request distribution: zipfian

recordcount=1000
operationcount=1000
workload=site.ycsb.workloads.CoreWorkload

readallfields=true

readproportion=0.95
updateproportion=0.05
scanproportion=0
insertproportion=0

requestdistribution=zipfian
```

Fig. 8. Configuration of *workload_b*: a read-heavy workload with 95% reads and 5% updates. This pattern reflects real-world systems where most operations are retrievals, such as search or recommendation engines.

```
GNU nano 7.2                                  workloadc
# Copyright (c) 2010 Yahoo! Inc. All rights reserved.
#
# Licensed under the Apache License, Version 2.0 (the "License"); you
# may not use this file except in compliance with the License. You
# may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
# implied. See the License for the specific language governing
# permissions and limitations under the License. See accompanying
# LICENSE file.

# Yahoo! Cloud System Benchmark
# Workload C: Read only
#   Application example: user profile cache, where profiles are constructed elsewhere (e.g., Hadoop)
#
#   Read/update ratio: 100/0
#   Default data size: 1 KB records (10 fields, 100 bytes each, plus key)
#   Request distribution: zipfian

recordcount=1000
operationcount=1000
workload=site.ycsb.workloads.CoreWorkload

readallfields=true

readproportion=1
updateproportion=0
scanproportion=0
insertproportion=0

requestdistribution=zipfian
```

Fig. 9. Configuration of *workload_c*: a pure read-only workload with 100% reads. This profile simulates static datasets such as user profiles or logs stored in NoSQL systems.

5.2. Benchmark results and runtime metrics by opscenter

We monitored the cluster using Apache OpsCenter and `nodetool` utilities to evaluate the runtime behavior of STCC. The experiments were conducted on a 24-node cluster during the execution of YCSB workloads A, B, and C. Metrics such as request latency, system load, disk usage, CPU activity, and throughput were captured in real time.

5.2.1. Write and read latency

Fig. 10 presents detailed latency percentiles for read and write operations executed under STCC with YCSB Workload-A (1,000,000 operations). The latency metrics, collected via OpsCenter, include minimum, median (50th), high (90th), tail (99th), and maximum values, offering a comprehensive view of response behavior. STCC maintains a tightly bounded latency profile for both reads and writes. In particular, the 99th percentile for reads remains below 63 ms, while the write latency exhibits significantly fewer tail spikes compared to static `QUORUM` settings. These results highlight the middleware's ability

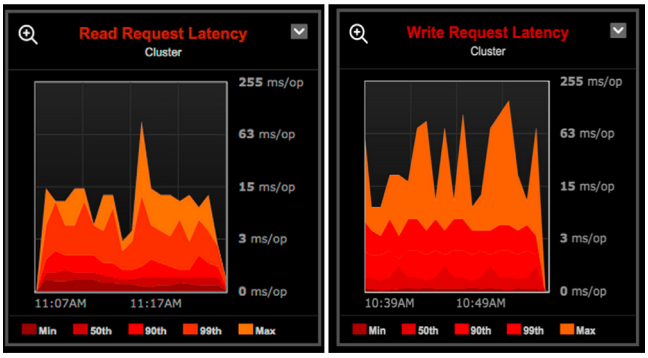


Fig. 10. Latency percentiles for read and write requests under STCC execution of YCSB Workload-A with 1,000,000 operations using `NetworkTopologyStrategy`. Data collected via OpsCenter includes `min`, `50th`, `90th`, `99th`, and `max` percentiles. STCC reduces tail latency and preserves consistent responsiveness.



Fig. 11. Cluster performance metrics captured via OpsCenter dashboard. Top-right: write request throughput. Bottom: disk usage, load, and per-node CPU activity.

to enforce consistency dynamically while reducing latency variability across percentile bands.

5.2.2. Cluster throughput and load distribution

Fig. 11 displays the overall write throughput reaching up to 3900 ops/sec under STCC enforcement. Despite enforcing strict consistency and freshness bounds, the system sustained high write rates due to adaptive replica selection and queue-aware dispatching. Load and CPU usage remained evenly distributed across nodes, indicating reduced hotspots compared to default Cassandra behavior.

5.2.3. Latency breakdown and alert management

Fig. 12 highlights real-time alerts and latency comparisons between reads and writes under varying load. STCC's temporal–causal enforcement produced predictable latency curves, while maintaining responsiveness under stress. Alerts were raised when node-specific write throughput exceeded expected thresholds, demonstrating the system's dynamic feedback mechanisms.

5.3. Monitoring YCSB workloads on Cassandra nodes

To facilitate transparent monitoring and performance evaluation, our implementation provides detailed runtime visibility into Cassandra's behavior using standard diagnostic tools and logs. Three key sources are employed:

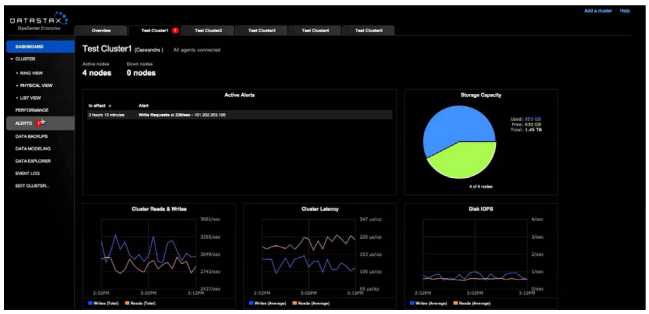
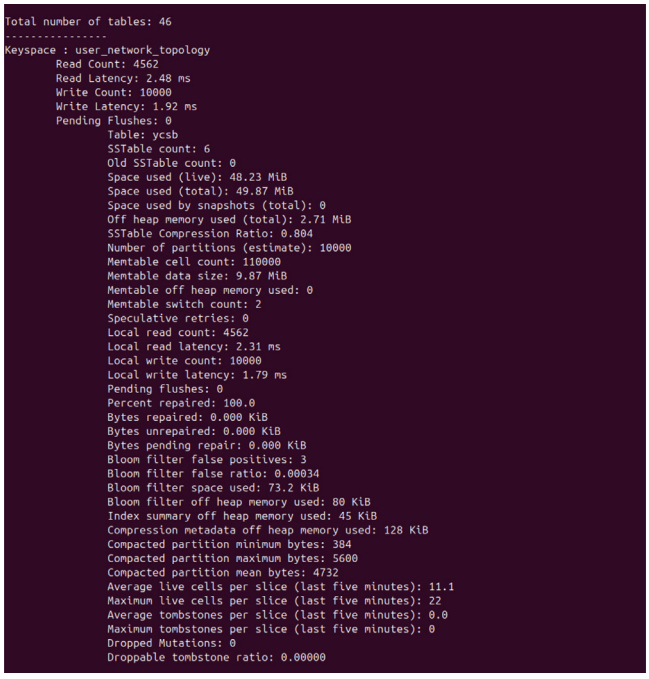


Fig. 12. Latency and request monitoring from a 4-node test cluster. The system raises alerts when write throughput exceeds threshold, ensuring operator awareness.



```
Total number of tables: 46
----------------
Keyspace : user_network_topology
        Read Count: 4562
        Read Latency: 2.48 ms
        Write Count: 10000
        Write Latency: 1.92 ms
        Pending Flushes: 0
                Table: ycsb
                SSTable count: 6
                Old SSTable count: 0
                Space used (live): 48.23 MiB
                Space used (total): 49.87 MiB
                Space used by snapshots (total): 0
                Off heap memory used (total): 2.71 MiB
                SSTable Compression Ratio: 0.804
                Number of partitions (estimate): 10000
                Memtable cell count: 110000
                Memtable data size: 9.87 MiB
                Memtable off heap memory used: 0
                Memtable switch count: 2
                Speculative retries: 0
                Local read count: 4562
                Local read latency: 2.31 ms
                Local write count: 10000
                Local write latency: 1.79 ms
                Pending flushes: 0
                Percent repaired: 100.0
                Bytes repaired: 0.000 KiB
                Bytes unrepaired: 0.000 KiB
                Bytes pending repair: 0.000 KiB
                Bloom filter false positives: 3
                Bloom filter false ratio: 0.00034
                Bloom filter space used: 73.2 KiB
                Bloom filter off heap memory used: 80 KiB
                Index summary off heap memory used: 45 KiB
                Compression metadata off heap memory used: 128 KiB
                Compacted partition minimum bytes: 384
                Compacted partition maximum bytes: 5600
                Compacted partition mean bytes: 4732
                Average live cells per slice (last five minutes): 11.1
                Maximum live cells per slice (last five minutes): 22
                Average tombstones per slice (last five minutes): 0.0
                Maximum tombstones per slice (last five minutes): 0
                Dropped Mutations: 0
                Droppable tombstone ratio: 0.00000
```

Fig. 13. A sample of `nodetool cfstats` output for the `ycsb` table of workload-A with quorum consistency under full replication.

5.3.1. nodetool cfstats — Table-level metrics

This command provides real-time statistics on the target table (e.g., `ycsb`), including:

- Read/Write counts and latencies (e.g., 2.48 ms reads, 1.92 ms writes)
- Memtable usage, SSTable counts, Bloom filter statistics
- Partition sizes and compression ratios

Command:

```
nodetool cfstats user_network_topology.
    ycsb > logs/structured_ycsb_data.csv
```

This output enables tracking resource usage, I/O patterns, and data layout efficiency during and after YCSB operations(see Fig. 13).

5.3.2. nodetool tpstats — Thread pool metrics

This command reveals low-level execution behavior, including:

- Stage activity (e.g., ReadStage, Compaction, MemtableFlush)
- Completed tasks, pending operations, and blocked queues

```
Pool Name                    Active Pending Completed Blocked All time blocked
ReadStage                         0       0      4562       0                0
CompactionExecutor                0       0     22415       0                0
MemtableReclaimMemory             0       0        75       0                0
PendingRangeCalculator            0       0         3       0                0
GossipStage                       0       0         0       0                0
SecondaryIndexManagement          0       0         1       0                0
HintsDispatcher                   0       0         0       0                0
MigrationStage                    0       0         4       0                0
MemtablePostFlush                 0       0        98       0                0
PerDiskMemtableFlushWriter_0      0       0        74       0                0
ValidationExecutor                0       0         0       0                0
Sampler                           0       0         0       0                0
ViewBuildExecutor                 0       0         0       0                0
MemtableFlushWriter               0       0        76       0                0
CacheCleanupExecutor              0       0         0       0                0
Native-Transport-Auth-Requests    0       0         0       0                0
Native-Transport-Requests         0       0     10345       0                0

Latencies waiting in queue (micros) per dropped message types
Message type                 Dropped     50%     95%     99%     Max
READ_RSP                           0    20.0    50.0    80.0    95.0
RANGE_REQ                          0     0.0     0.0     0.0     0.0
PAXOS_COMMIT_REQ                   0    10.0    25.0    35.0    40.0
MUTATION_REQ                       0    18.0    42.0    63.0    75.0
VALIDATION_REQ                     0     0.0     0.0     0.0     0.0
SCHEMA_VERSION_REQ                 0     0.0     0.0     0.0     0.0
READ_REQ                           0    21.0    47.0    70.0    81.0
PAXOS_PREPARE_REQ                  0    12.0    23.0    31.0    39.0
SNAPSHOT_REQ                       0     0.0     0.0     0.0     0.0
BATCH_STORE_REQ                    0     9.0    18.0    24.0    28.0
```

Fig. 14. A sample of `nodetool tpstats` output for the `ycsb` table of workload-A with quorum consistency under full replication.

```
2025-08-04 18:04:31:855 0 sec: 0 operations; est completion in 106751991167300 days 15 hours
[OVERALL], RunTime(ms), 365
[OVERALL], Throughput(ops/sec), 15.0
[TOTAL_GCS_G1_Young_Generation], Count, 1
[TOTAL_GC_TIME_G1_Young_Generation], Time(ms), 4
[TOTAL_GC_TIME_%_G1_Young_Generation], Time(%), 1.095890410958904
[TOTAL_GCS_G1_Old_Generation], Count, 0
[TOTAL_GC_TIME_G1_Old_Generation], Time(ms), 0
[TOTAL_GC_TIME_%_G1_Old_Generation], Time(%), 0.0
[TOTAL_GCs], Count, 1
[TOTAL_GC_TIME], Time(ms), 4
[TOTAL_GC_TIME_%], Time(%), 1.095890410958904
```

Fig. 15. A sample YCSB benchmark output showing throughput and GC statistics of running workload-A with quorum consistency under full replication.

- Dropped message analysis with queue latency quantiles (50%, 95%, 99%)

Command:

```
nodetool tpstats
```

This is particularly useful for observing backpressure or execution bottlenecks during high-throughput scenarios(see Fig. 14).

5.3.3. YCSB runtime logs

The YCSB benchmark tool outputs real-time information including:

- Throughput (operations per second)
- Benchmark run time in milliseconds
- Garbage Collection (GC) activity: count and duration

Command:

```
./bin/ycsb run cassandra-cql -s -P
    workloads/workloada -p hosts
    =192.168.1.100 > logs/
    structured_ycsb_data.csv
```

Sample Output:

```
[OVERALL], RunTime(ms), 365
[OVERALL], Throughput(ops/sec), 15.0
[TOTAL_GCs], Count, 1
[TOTAL_GC_TIME], Time(ms), 4
[TOTAL_GC_TIME_
```

These metrics help verify the stability and overhead of the benchmark environment. All generated results can be logged and compared across different YCSB workloads, consistency levels, replication factors, or middleware configurations(see Fig. 15).

These metrics help verify the stability and overhead of the benchmark environment. All generated results can be logged and compared across different settings, such as consistency levels, replication factors, or middleware configurations.

5.4. Power and frequency monitoring via Linux governors and PDU readings

Algorithm 2: Energy-Aware YCSB Benchmarking with Consistency Variants and PDU Monitoring

Input: Workload Type $w \in \{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$, Consistency Level $c \in \{\mathtt{ALL}, \mathtt{QUORUM}, \mathtt{ONE}\}$, Frequency Governor f_g, Node Load Threshold θ

Output: Execution Time, Throughput, CPU Usage, Total Energy

```
SetFrequencyGovernor(f_g)      // e.g., powersave, performance, etc.
nodes ← SelectNodesUnderThreshold(θ)
if nodes = ∅ then
    exit with error: "No healthy nodes under threshold"
StartMonitoring()              // Start dstat and power logger (e.g., power-monitor.sh)
LoadYCSB(w, c, nodes)          // Invoke script based on c: default_one.py, etc.
RunYCSB(w, c, nodes)           // Same script continues to run workload phase
StopMonitoring()
results ← ComputeStatistics(power_logs, dstat_log, ycsb_output)
LogSummary(results)
```

Algorithm 2 Interpretation. The above pseudocode represents the workflow for running energy-aware consistency benchmarks across Cassandra clusters. Each pair of workload ($w \in \{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$) and consistency level ($c \in \{\mathtt{ALL}, \mathtt{QUORUM}, \mathtt{ONE}\}$) corresponds to a dedicated runner script (e.g., `default_all.py`, `default_quorum.py`, or `default_one.py`). These scripts first perform data loading (via `ycsb load`) and then execute the actual benchmark (via `ycsb run`) on the selected nodes. Frequency governors are applied system-wide before each run using `cpupower`, and runtime metrics (power, CPU usage, latency) are captured using `dstat` and a programmable PDU. Finally, all measurements are aggregated into a summary log for post-analysis. The subsequent sections provide a comprehensive description of how CPU frequency governors are programmatically controlled and how real-time power and CPU usage measurements are captured using programmable PDU tools and system utilities.

5.4.1. Dynamic frequency scaling (DVFS) modes

We employed five standard Linux frequency governors to control CPU behavior during workload execution:

- `ondemand` — Dynamically adjusts frequency based on current load.
- `conservative` — Similar to `ondemand` but increases frequency more gradually.
- `performance` — Forces the CPU to operate at maximum frequency.
- `powersave` — Forces the CPU to operate at the lowest available frequency.
- `userspace` — Allows manual control over frequency.

These modes were set using the `cpupower` utility:

Governor Inspection and Setting:

Listing 1 Querying and setting Linux CPU frequency governors

```
# Show current governor and frequency
cpupower frequency-info

# Set all cores to a specific governor
cpupower frequency-set -g performance
cpupower frequency-set -g powersave
cpupower frequency-set -g ondemand
cpupower frequency-set -g conservative

# Enable manual frequency control
cpupower frequency-set -g userspace
cpupower frequency-set -u 1.8GHz
cpupower frequency-set -d 1.8GHz
cpupower frequency-set -f 1.8GHz
```

Sample output from `cpupower frequency-info`:

Listing 2 Output example from cpupower frequency-info

```
analyzing CPU 0:
driver: intel_pstate
current policy: frequency should be
    within 800 MHz and 2.8 GHz.
The governor "userspace" may decide
    which speed to use.
current CPU frequency is 1.8 GHz
```

5.4.2. PDU-based power and energy monitoring

A programmable PDU was connected to all nodes of the Cassandra cluster to evaluate energy consumption in real time. The PDU provides power readings (in Watts) via SNMP or USB interfaces. During each YCSB benchmark run, power values were sampled every second, enabling accurate energy computation based on runtime integration.

In addition to power, CPU usage was monitored using the `dstat` tool to correlate system activity with energy patterns.

Example SNMP command for live power polling:

Listing 3 Polling power from PDU via SNMP

```
snmpwalk -v2c -c public <PDU_IP>
    .1.3.6.1.4.1.318.1.1.12.1.16.0
```

CPU usage and system activity were logged using:

Listing 4 Real-time CPU and system monitoring

```
dstat -cdnm --output logs/
    ycsb_energy_data_runtime.csv
 1 300
```

5.4.3. Energy and power calculation

Dynamic energy consumption for each workload was computed by integrating power over the execution time:

$$E_{\text{dynamic}} = (P_{\text{avg,load}} - P_{\text{idle}}) \times t_{\text{exec}}$$

Where:

- $P_{\text{avg,load}}$ is the average power during workload execution.
- P_{idle} is the average power during system idle state.
- t_{exec} is the measured runtime of the workload.

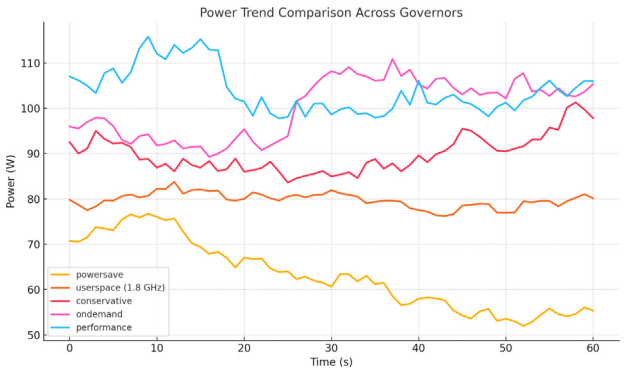


Fig. 16. Power profile recorded during YCSB Workload-A (1M operations) using full replication and `QUORUM` consistency. Data collected via PDU over a 60-second monitoring window.

6. Use case and validation scenario

To demonstrate the practical applicability of the software, we deployed it on a 24-node Apache Cassandra 4.0 cluster across three geographically separated data centers. Each node was interconnected via high-speed links and configured with custom replication strategies to emulate a realistic multi-region cloud deployment [34].

We executed three standard YCSB workloads (`a`, `b`, and `c`) under both native consistency levels — `ALL`, `QUORUM`, `ONE` — and the proposed STCC middleware. CPU frequency governors were set using `cpupower` to evaluate the system under various DVFS modes: `powersave`, `conservative`, `ondemand`, `performance`, and `userspace`.

To monitor resource behavior, we collected:

- Power consumption via SNMP-enabled PDU, sampled every second.
- CPU usage and disk/network I/O using `dstat`.
- Throughput and latency using `nodetool` and `OpsCenter`.

Fig. 16 illustrates a power profile captured during the execution of YCSB Workload-A with full replication and `QUORUM` consistency. The graph highlights the energy consumption patterns across different CPU governors. While `powersave` minimized power draw, `performance` achieved the highest throughput at the cost of increased energy. Intermediate policies like `ondemand` offered a balance between the two.

Key experimental findings include:

- **Reduced staleness**: STCC lowered stale read ratios by up to 25% compared to native static consistency levels.
- **Energy–performance variation**: Energy consumption differed by over 30% across DVFS modes for the same workload and consistency configuration.
- **Practical tunability**: The suite enables users to identify optimal CPU-frequency–consistency configurations aligned with latency, freshness, or energy targets.

These results validate the middleware's ability to enforce adaptive consistency policies while enabling reproducible, energy-aware experimentation. The software provides a flexible platform for benchmarking trade-offs in real-world Cassandra deployments and for guiding data-driven configuration decisions in industry and research contexts.

7. Conclusion and future work

This paper presented two complementary contributions toward scalable and energy-efficient cloud storage.

First, we revisited and improved the Strict Timed Causal Consistency (STCC) model previously introduced in [19], deploying it

in a production-grade Cassandra cluster via our custom middleware and benchmarking framework. Our experiments confirm that STCC enables enforcement of client-centric consistency (e.g., MR, MW, RYW, WFR) without sacrificing system throughput, by offloading dependency tracking to the middleware layer.

Second, we conducted a comparative analysis of energy consumption under Cassandra's native consistency levels — ALL, QUORUM, and ONE — without STCC. Using a programmable PDU and system-level instrumentation (e.g., `dstat`), we quantified the Energy–performance trade-offs of each consistency level under DVFS and governor-controlled environments.

Future work includes dynamic Δ-tuning based on workload characteristics, extending the STCC model to support hybrid cloud deployments, and integrating the benchmark suite with other NoSQL platforms such as MongoDB and HBase.

CRediT authorship contribution statement

Hesam Nejati SharifAldin: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Farnoush Nayebi Pour:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] James Manyika, Michael Chui, Brad Brown, Jacques Bughin, Richard Dobbs, Charles Roxburgh, Angela Hung Byers, Big Data: the Next Frontier for Innovation, Competition, and Productivity, McKinsey & Co., p. 2011.

[2] Shui Yu, Meng Liu, Wanchun Dou, Xiting Liu, Sanming Zhou, Networking for big data: A survey, IEEE Commun. Surv. Tutor. 19 (1) (2017) 531–549, http://dx.doi.org/10.1109/COMST.2016.2610963.

[3] Gabriel Heleno Gonçalves da Silva, Maristela Holanda, Aleteia Araujo, Data replication policy in a cloud computing environment, in: Proceedings of the 11th Iberian Conference on Information Systems and Technologies, CISTI, 2016, pp. 1–6, http://dx.doi.org/10.1109/CISTI.2016.7521383.

[4] Jinyuan Li, David Mazieres, Beyond one-third faulty replicas in Byzantine fault tolerant systems, in: Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation, NSDI, USENIX Association, 2007, p. 10.

[5] Faheem Zafar, Abid Khan, Saif Ur Rehman Malik, Mansoor Ahmed, Adeel Anjum, Majid Iqbal Khan, Nadeem Javed, Masoom Alam, Fuzel Jamil, A survey of cloud computing data integrity schemes: Design challenges, taxonomy and future trends, Comput. Secur. 65 (2017) 29–49, http://dx.doi.org/10.1016/j.cose.2016.10.006.

[6] Werner Vogels. Eventually consistent, Commun. ACM 52 (1) (2009) 40–44, ACM, New York, NY, USA.

[7] Tobias Pfandzelter, Nils Japke, Trever Schirmer, Jonathan Hasenburg, David Bermbach, Managing data replication and distribution in the fog with FReD, Softw.: Pr. Exp. 53 (10) (2023) 1958–1981, http://dx.doi.org/10.1002/spe.3173.

[8] Eric Brewer, CAP twelve years later: How the "rules" have changed, IEEE Comput. 45 (2) (2012) 23–29, http://dx.doi.org/10.1109/MC.2012.37.

[9] Seth Gilbert, Nancy Lynch, Perspectives on the CAP theorem, IEEE Comput. 45 (2) (2012) 30–36, http://dx.doi.org/10.1109/MC.2011.389.

[10] Rachid Guerraoui, Matej Pavlovic, Dragos-Adrian Seredinschi, Trade-offs in replicated systems, IEEE Data Eng. Bull. 39 (1) (2016) 14–26.

[11] Prince Mahajan, Lorenzo Alvisi, Mike Dahlin, et al., Consistency, availability, and convergence, Univ. Tex. Austin Tech. Rep. 11 (2011) 158.

[12] Leslie Lamport, Time and clocks, and the ordering of events in a distributed system, in: Concurrency: The Works of Leslie Lamport, Association for Computing Machinery, New York, NY, USA, 2019, pp. 179–196, http://dx.doi.org/10.1145/3335772.3335934.

[13] Douglas B. Terry, Alan J. Demers, Karin Petersen, Mike J. Spreitzer, Marvin M. Theimer, Brent B. Welch, Session guarantees for weakly consistent replicated data, in: Proceedings of the 3rd International Conference on Parallel and Distributed Information Systems, PDIS, 1994, pp. 140–149, http://dx.doi.org/10.1109/PDIS.1994.331722.

[14] Qin Liu, Guojun Wang, Jie Wu, Consistency as a service: Auditing cloud consistency, IEEE Trans. Netw. Serv. Manag. 11 (1) (2014) 25–35, http://dx.doi.org/10.1109/TNSM.2013.122613.130411.

[15] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno Preguiça, Rodrigo Rodrigues, Making Geo-Replicated systems fast as possible, consistent when necessary, in: Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation, OSDI'12, USENIX Association, 2012, pp. 265–278.

[16] Ta-Yuan Hsu, Ajay D. Kshemkalyani, Min Shen, Causal consistency algorithms for partially replicated and fully replicated systems, Future Gener. Comput. Syst. 86 (2018) 1118–1133, http://dx.doi.org/10.1016/j.future.2017.08.049.

[17] Lucas Ferreira Da Silva, João V.F. Lima, An evaluation of Cassandra NoSQL database on a low-power cluster, in: Proceedings of the 2021 International Symposium on Computer Architecture and High Performance Computing Workshops, SBAC-PADW, 2021, pp. 9–14, http://dx.doi.org/10.1109/SBAC-PADW53941.2021.00012.

[18] Yunxiao Du, Zichen Xu, Kanqi Zhang, Jie Liu, Christopher Stewart, Jiacheng Huang, Cost-effective strong consistency on scalable geo-diverse data replicas, IEEE Trans. Cloud Comput. 11 (2) (2023) 1764–1776, http://dx.doi.org/10.1109/TCC.2022.3161297.

[19] Hesam Nejati Sharif Aldin, Hossein Deldari, Mohammad Hossein Moattar, Mostafa Razavi Ghods, Strict timed causal consistency as a hybrid consistency model in the cloud environment, Future Gener. Comput. Syst. 105 (2020) 259–274, http://dx.doi.org/10.1016/j.future.2019.11.009.

[20] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, Russell Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC'10, Association for Computing Machinery, Indianapolis, Indiana, USA, 2010, pp. 143–154, http://dx.doi.org/10.1145/1807128.1807152.

[21] Francisco J. Torres-Rojas, Mustaque Ahamad, Michel Raynal, Timed consistency for shared distributed objects, in: Proceedings of the Eighteenth Annual ACM Symposium on Principles of Distributed Computing, PODC'99, Association for Computing Machinery, Atlanta, Georgia, USA, 1999, pp. 163–172, http://dx.doi.org/10.1145/301308.301350.

[22] Francisco J. Torres-Rojas, Esteban Meneses, Convergence through a weak consistency model: Timed causal consistency, CLEI Electron. J. 8 (2) (2005) 2–1.

[23] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, David G. Andersen, Don't settle for eventual: Scalable causal consistency for wide-area storage with COPS, in: Proceedings of the 23rd ACM Symposium on Operating Systems Principles, SOSP, 2011, pp. 1–16, http://dx.doi.org/10.1145/2043556.2043593.

[24] Min Shen, Ajay D. Kshemkalyani, Ta-Yuan Hsu, Causal consistency for geo-replicated cloud storage under partial replication, in: Proceedings of the 2015 IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPSW, 2015, pp. 509–518, http://dx.doi.org/10.1109/IPDPSW.2015.68.

[25] Yue Li, Energy Measurements for Cassandra Cloud Storage System: Exploring and Improving Energy-Consistency Tradeoff (Master's Thesis), University of Florida and Technical University of Munich, 2013.

[26] H.E. Chihoub, S. Ibrahim, Y. Li, G. Antoniu, M.S. Pérez, L. Bougé, Exploring energy-consistency trade-offs in Cassandra cloud storage system, in: Proc. 27th Int. Symp. Computer Architecture and High Performance Computing, SBAC-PAD, 2015, pp. 146–153, http://dx.doi.org/10.1109/SBAC-PAD.2015.28.

[27] S. Ibrahim, T.-D. Phan, A. Carpen-Amarie, H.-E. Chihoub, D. Moise, G. Antoniu, Governing energy consumption in hadoop through CPU frequency scaling: An analysis, Future Gener. Comput. Syst. 54 (2016) 219–232, http://dx.doi.org/10.1016/j.future.2015.01.005.

[28] D. Mahajan, Z. Zong, Energy efficiency analysis of query optimizations on MongoDB and Cassandra, in: Proc. 8th Int. Green and Sustainable Computing Conf., IGSC, 2017, pp. 1–6, http://dx.doi.org/10.1109/IGCC.2017.8323581.

[29] H. Nejati Sharif Aldin, F. Nayebi Pour, From watts to writes: the role of frequency, replication strategies and consistency levels in performance and energy-aware cassandra cluster operations, Future Gener. Comput. Syst. 176 (2026) 108210, http://dx.doi.org/10.1016/j.future.2025.108210.

[30] S.R. Gari, A.K. Reddy, Performance evaluation of Cassandra in AWS environment: An experiment, 2017, Unpublished manuscript.

[31] C. Gomes, E. Tavares, M.N.O. Junior, B. Nogueira, Cloud storage availability and performance assessment: A study based on NoSQL DBMS, J. Supercomput. 78 (2) (2022) 2819–2839.

[32] C. Gomes, M.N. de O. Junior, B. Nogueira, P. Maciel, E. Tavares, NoSQL-based storage systems: Influence of consistency on performance, availability and energy consumption, J. Supercomput. 79 (18) (2023) 21424–21448.

[33] A. Rafay, Performance analysis/measurements with Cassandra and HBase, 2023, Unpublished report.

[34] H. Nejati Sharif Aldin, F. Nayebi Pour, An industrial benchmarking suite for energy–performance–consistency trade-offs in apache Cassandra with middleware-enforced timed causal consistency and static consistency levels, 2025, http://dx.doi.org/10.5281/zenodo.16742599, Zenodo, Version v1.0. 0.



Hesam Nejati SharifAldin received his Master's degree in software engineering from the Islamic Azad University of Mashhad, Iran, in 2017. He is pursuing a Ph.D. degree from the School of Computer Science, University College Dublin. His research interests include consistency in distributed systems, cloud computing, parallel computing, high-performance computing, and energy-efficient computing.



Farnoush Nayebi Pour received her Bachelor's degree in software engineering from the Islamic Azad University of Mashhad, Iran, in 2020. Her research interests include cloud computing, energy efficiency in IoT communication, and IoT protocol sensors.